

Projektarbeit: Erarbeitungs-/Reflexionsphase

Internationale Hochschule Duales Studium
Studiengang:
Informatik

Thema

**Data-Mart-Erstellung in
SQL (DLBDSPBDM01_D)**

Furkan Demir
Matrikelnummer: IU14138041
Jahnstraße 17, 55257 Budenheim

Betreuende Person: Anna Androvitsanea

Inhaltsverzeichnis

ERARBEITUNGS-/REFLEXIONSPHASE – BUCHTAUSCH-APP	1
EINLEITUNG UND ZIEL	1
TECHNISCHE UMGEBUNG UND DATENBANKVERBINDUNG	1
VORGEHENSWEISE BEI DER IMPLEMENTIERUNG	3
DATENBANKSTRUKTUR UND NORMALISIERUNG	3
ER-MODELL.....	6
DOKUMENTATION DER TABELLEN UND BEZIEHUNGEN.....	7
TABELLE BENUTZER	7
TABELLE GENRE	7
TABELLE AUTOR	8
TABELLE SPRACHE.....	9
TABELLE VERLAG	10
TABELLE STANDORT	10
TABELLE BUCH	11
TABELLE ANGEBOT	12
TABELLE ZEITSLOT.....	13
TABELLE BUCHUNGSLISTE	14
TABELLE RESERVIERUNG	15
TABELLE AUSLEIHE.....	16
TABELLE BEWERTUNG	17
TABELLE ZAHLUNG	17
TABELLE RECHNUNG	18
DUMMY-DATEN UND DATENKONSISTENZ.....	19
TESTFÄLLE UND KONTROLLABFRAGEN	21
TESTFALL 1 – ANZEIGE VERFÜGBARER BUCHANGEBOTE	21
TESTFALL 2 – NACHVERFOLGUNG DES AUSLEIH- UND ZAHLUNGSPROZESSES	22
TESTFALL 3 – ANZEIGE VON BEWERTUNGEN	23
TESTFALL 4 – ANZEIGE VON ZAHLUNGEN	23
TESTFALL 5 – AKTUALISIERUNG EINER AUSLEIHE	24
PROBLEME, FEHLERANALYSE UND LÖSUNGEN.....	25
ZUSAMMENFASSUNG DER IMPLEMENTIERUNG	26
FAZIT.....	27

Tabellenverzeichnis

Tabelle 1 DB Verbindung.....	1
Tabelle 2 Implementierungsreihenfolge	3
Tabelle 3 Übersicht der Datenbanktabellen und Schlüsselattribute	5
Tabelle 4 Probleme, Fehleranalyse	25

Abbildungsverzeichnis

Abbildung 1 XAMPP mit gestartetem Datenbankdienst	2
Abbildung 2 DBeaver-Verbindungseinstellungen mit localhost und Port 3307	2
Abbildung 3 DBeaver Databases Buchtausch_app	3
Abbildung 4: Entity-Relationship-Modell ERM	6
Abbildung 5 Benutzer	7
Abbildung 6 select Aufruf Benutzer	7
Abbildung 7 Genre	8
Abbildung 8 select Aufruf Genre.....	8
Abbildung 9 Autor	8
Abbildung 10 select Aufruf Autor.....	9
Abbildung 11 Sprache	9
Abbildung 12 select Aufruf Sprache.....	9
Abbildung 13 Verlag	10
Abbildung 14 select Aufruf Verlag.....	10
Abbildung 15 Standort	11
Abbildung 16 select Aufruf Standort	11
Abbildung 17 Buch.....	12
Abbildung 18 select Aufruf Buch	12
Abbildung 19 Angebot	13
Abbildung 20 select Aufruf Angebot	13
Abbildung 21 Zeitslot	14
Abbildung 22 select Aufruf zeitslot	14
Abbildung 23 Buchungsliste	14
Abbildung 24 select Aufruf Buchungsliste	15
Abbildung 25 reservierung	15
Abbildung 26 select Aufruf Reservierung	15
Abbildung 27 Ausleihe.....	16
Abbildung 28 select Aufruf Ausleihe	16
Abbildung 29 Bewertung	17
Abbildung 30 select Aufruf Bewertung	17
Abbildung 31 Zahlung	18
Abbildung 32 select Aufruf Zahlung	18
Abbildung 33 Rechnung	19
Abbildung 34 select Aufruf Rechnung	19
Abbildung 35 Dummy-Daten und Datenkonsistenz	20
Abbildung 36 Update Angebotsdaten	21
Abbildung 37 Test 1	22
Abbildung 38 Test 2	23
Abbildung 39 Test 3	23

Abbildung 40 Test 4.....	24
Abbildung 41 Test 5 Vorher	25
Abbildung 42 Test 5 Nachher	25

Erarbeitungs-/Reflexionsphase – Buchtausch-App

Einleitung und Ziel

In der zweiten Phase wurde das in der Konzeptionsphase entwickelte relationale Datenmodell der Buchtausch-App technisch umgesetzt. Ziel war es, die zuvor modellierten Entitäten und Beziehungen in SQL-Tabellen zu übertragen, geeignete Primär- und Fremdschlüssel zu definieren und die Datenbank mit sinnvollen Dummy-Daten zu befüllen.

Die Buchtausch-App bildet einen lokalen Ausleihprozess ab. Benutzer können Bücher anbieten, andere Benutzer können diese Angebote vormerken, reservieren und ausleihen. Darüber hinaus werden Bewertungen, Zahlungen und Rechnungen verwaltet.

Technische Umgebung und Datenbankverbindung

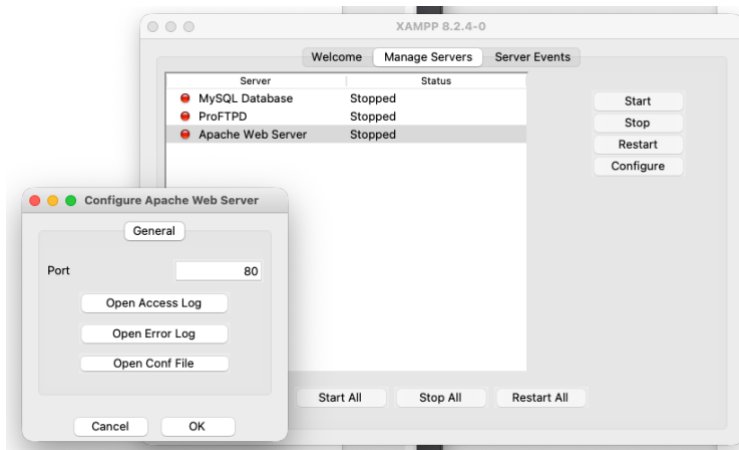
Für die Umsetzung der Datenbank wurde eine lokale Entwicklungsumgebung erstellt. Als Serverumgebung kam XAMPP zum Einsatz. Über XAMPP wurde der Datenbankdienst gestartet, damit lokal auf die MariaDB-/MySQL-Datenbank zugegriffen werden konnte. In der XAMPP-Oberfläche wurde dafür der Datenbankserver gestartet und der verwendete Port kontrolliert.

Anschließend wurde in DBeaver eine neue Datenbankverbindung eingerichtet. Als Verbindungstyp wurde MariaDB verwendet. Die Verbindung erfolgte über den lokalen Host localhost und den Port 3307. Als Benutzer wurde root verwendet. Ein Passwort wurde nicht eingetragen, da die lokale XAMPP-Datenbank ohne Passwort konfiguriert war.

Tabelle 1 DB Verbindung

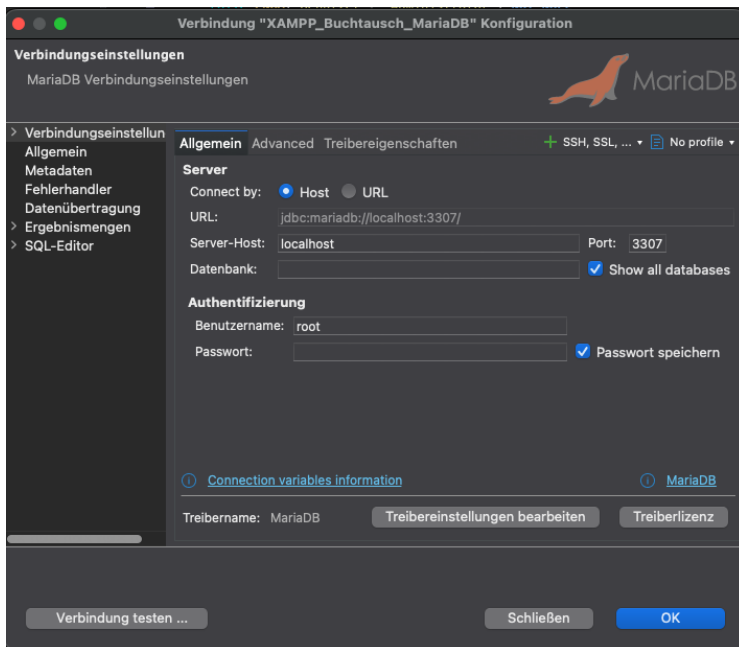
Feld	Wert
Server-Host	localhost
Port	3307
Datenbank	leer lassen / danach buchtausch_app auswählen
Benutzername	root
Passwort	leer lassen

Abbildung 1 XAMPP mit gestartetem Datenbankdienst



Quelle: Eigene Darstellung 1

Abbildung 2 DBeaver-Verbindungseinstellungen mit localhost und Port 3307

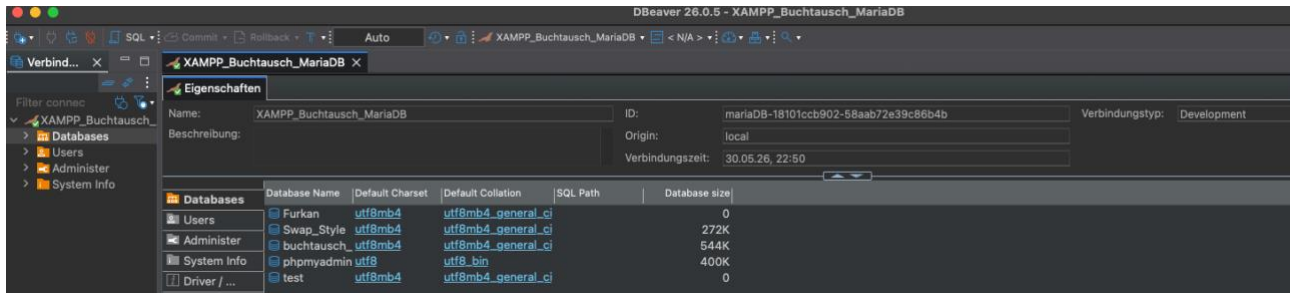


Quelle: Eigene Darstellung 2

```
SHOW DATABASES;  
CREATE DATABASE IF NOT EXISTS buchtausch_app  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_general_ci;  
USE buchtausch_app;  
SELECT DATABASE();
```

Der Zeichensatz utf8mb4 wurde gewählt, damit Umlaute, Sonderzeichen und internationale Zeichen korrekt gespeichert werden können. Die Kollation utf8mb4_general_ci sorgt dafür, dass Textvergleiche weitgehend unabhängig von Groß- und Kleinschreibung erfolgen.

Abbildung 3 DBeaver Databases Buchtausch_app



Quelle: Eigene Darstellung 3

Vorgehensweise bei der Implementierung

Die Implementierung erfolgte schrittweise. Zunächst wurden die Stammdatentabellen erstellt, da diese keine oder nur wenige Abhängigkeiten aufweisen. Anschließend wurden die abhängigen Tabellen mit Fremdschlüsseln angelegt. Dieser Ansatz verhindert Fremdschlüssel-Fehler während der Tabellenerstellung.

Tabelle 2 Implementierungsreihenfolge

Reihenfolge	Tabellen	Begründung
1	benutzer, genre, autor, sprache, verlag, standort	Diese Tabellen bilden grundlegende Stammdaten und werden später von anderen Tabellen referenziert.
2	buch	Buch verweist auf Sprache, Autor, Genre und Verlag.
3	angebot, zeitslot, buchungsliste	Diese Tabellen verbinden Bücher, Benutzer, Standorte und mögliche Abholzeiten.
4	reservierung, ausleihe, zahlung, rechnung	Diese Tabellen bilden den Ablauf von Reservierung bis Rechnung ab.
5	bewertung	Bewertungen verbinden Benutzer und Bücher.

Datenbankstruktur und Normalisierung

Die Datenbankstruktur der Buchtausch-App wurde auf Grundlage der funktionalen und nicht funktionalen Anforderungen aus der Konzeptionsphase umgesetzt. Ziel war es, eine relationale Datenbank zu erstellen, die den gesamten Prozess der Buchausleihe nachvollziehbar abbildet. Dazu gehören die Registrierung von Benutzern, das Erfassen und Anbieten von Büchern, die Suche nach verfügbaren Angeboten sowie die Reservierung, Ausleihe, Rückgabe, Bewertung und Zahlungsabwicklung.

Die funktionalen Anforderungen wurden durch mehrere miteinander verknüpfte Tabellen umgesetzt. Benutzer können Bücher erfassen und als Angebote bereitstellen. Zu jedem Angebot können Informationen wie Verfügbarkeit, Leihdauer, Standort, Zeitslot, Versandoption und Zustand

gespeichert werden. Außerdem können Benutzer Angebote in eine Buchungsliste aufnehmen, Reservierungen erstellen, Bücher ausleihen und anschließend Bewertungen abgeben. Zahlungen und Rechnungen werden ebenfalls als eigene Tabellen abgebildet, damit der Ausleihprozess vollständig dokumentiert werden kann.

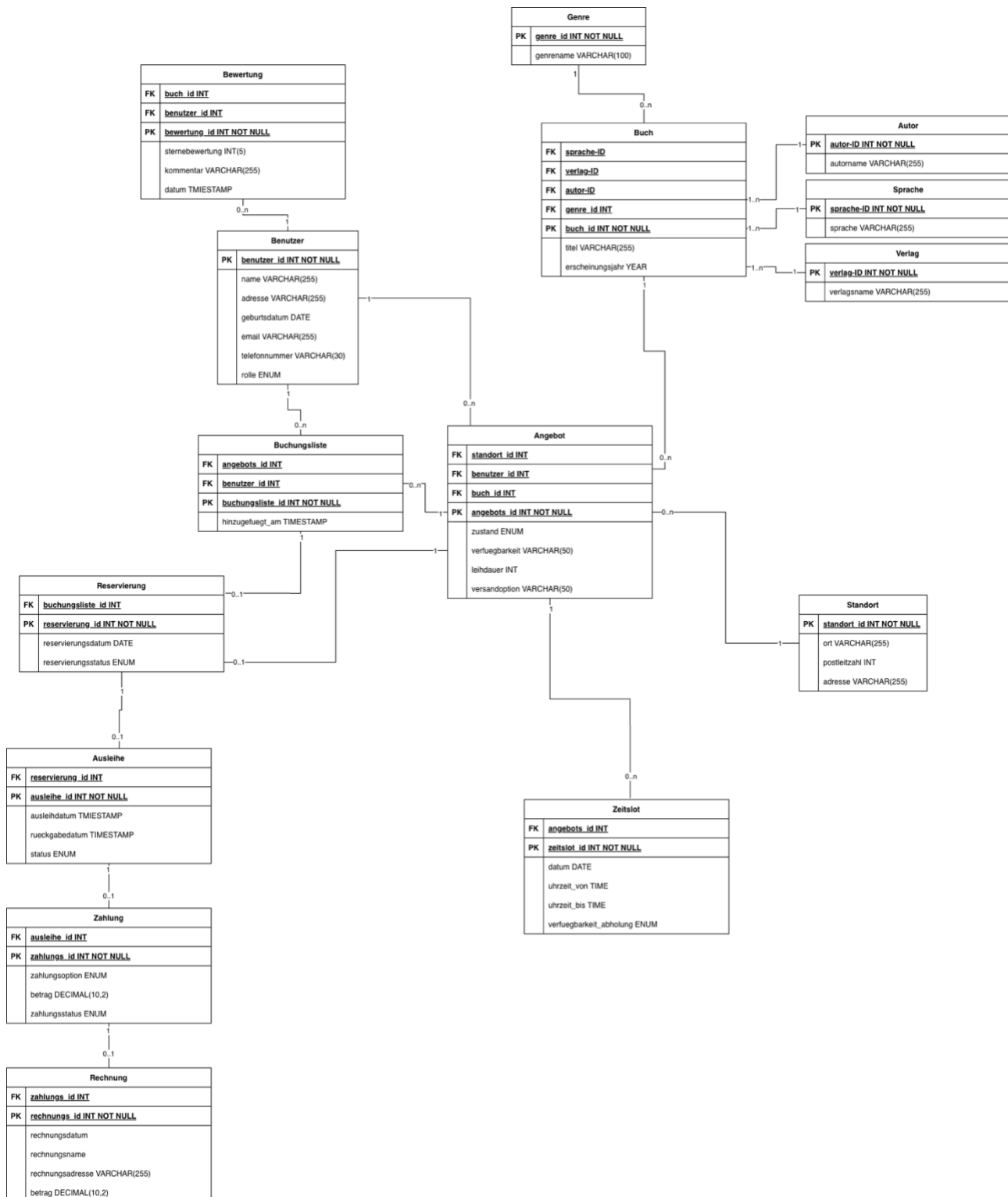
Zur Umsetzung der nicht funktionalen Anforderungen wurde die Datenbank normalisiert aufgebaut. Wiederkehrende Informationen wie Autor, Genre, Sprache und Verlag wurden nicht direkt in der Tabelle buch gespeichert, sondern in eigene Tabellen ausgelagert. Dadurch werden doppelte Daten vermieden und die Datenbank bleibt übersichtlich und konsistent. Die Beziehungen zwischen den Tabellen werden durch Primär- und Fremdschlüssel abgesichert. Dadurch wird sichergestellt, dass zum Beispiel eine Ausleihe nur zu einer bestehenden Reservierung und eine Rechnung nur zu einer vorhandenen Zahlung erstellt werden kann.

Tabelle 3 Übersicht der Datenbanktabellen und Schlüsselattribute

Tabelle	Primärschlüssel	Wichtige Fremdschlüssel	Funktion
benutzer	benutzer_id	-	Speichert Benutzer und Administratoren.
genre	genre_id	-	Speichert Buchgenres.
autor	autor_id	-	Speichert Autoren.
sprache	sprache_id	-	Speichert Sprachen.
verlag	verlag_id	-	Speichert Verlage.
standort	standort_id	-	Speichert Abhol- und Versandstandorte.
buch	buch_id	sprache_id, autor_id, genre_id, verlag_id	Speichert grundlegende Buchinformationen.
angebot	angebots_id	buch_id, benutzer_id, standort_id	Verbindet Buch, Anbieter und Standort.
zeitslot	zeitslot_id	angebots_id	Speichert mögliche Abholzeiten.
buchungsliste	buchungsliste_id	benutzer_id, angebots_id	Speichert vorgemerkte Angebote.
reservierung	reservierung_id	buchungsliste_id	Speichert Reservierungen.
ausleihe	ausleihe_id	reservierung_id	Speichert Ausleihvorgänge.
bewertung	bewertung_id	benutzer_id, buch_id	Speichert Bewertungen.
zahlung	zahlungs_id	ausleihe_id	Speichert Zahlungsdaten.
rechnung	rechnungs_id	zahlungs_id	Speichert Rechnungen.

ER-Modell

Abbildung 4: Entity-Relationship-Modell ERM



Quelle: Eigene Darstellung 4

Dokumentation der Tabellen und Beziehungen

Tabelle Benutzer

Speichert Benutzer und Administratoren.

```
CREATE TABLE benutzer (  
  benutzer_id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  adresse VARCHAR(255),  
  geburtsdatum DATE,  
  e_mail VARCHAR(100) NOT NULL UNIQUE,  
  telefonnummer VARCHAR(30),  
  rolle ENUM('Benutzer', 'Administrator') NOT NULL  
);
```

```
DESCRIBE benutzer;
```

Abbildung 5 Benutzer

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	benutzer_id	int(11)	NO	PRI	[NULL]	auto_increment
2	name	varchar(100)	NO		[NULL]	
3	adresse	varchar(255)	YES		[NULL]	
4	geburtsdatum	date	YES		[NULL]	
5	e_mail	varchar(100)	NO	UNI	[NULL]	
6	telefonnummer	varchar(30)	YES		[NULL]	
7	rolle	enum('Benutzer', 'Administrator')	NO		[NULL]	

Quelle: Eigene Darstellung 5

```
SELECT * from benutzer;
```

Abbildung 6 select Aufruf Benutzer

	benutzer_id	AZ name	AZ adresse	AZ geburtsdatum	AZ e_mail	AZ telefonnummer	AZ rolle
1	1	Max Mustermann	Musterstraße 1, 55257 Budenheim	29.06.1997	max.mustermann@example.com	015100000001	Benutzer
2	2	Anna Beispiel	Buchweg 2, 55257 Budenheim	13.03.2000	anna.beispiel@example.com	015100000002	Administrator
3	3	Lukas Weber	Lesestraße 3, 55257 Budenheim	06.07.2008	lukas.weber@example.com	015100000003	Benutzer
4	4	Mia Schneider	Tauschallee 4, 55257 Budenheim	27.01.1992	mia.schneider@example.com	015100000004	Benutzer
5	5	Noah Fischer	Romanweg 5, 55257 Budenheim	18.05.1997	noah.fischer@example.com	015100000005	Benutzer
6	6	Lea Wagner	Bibliotheksweg 6, 55257 Budenheim	27.02.1983	lea.wagner@example.com	015100000006	Benutzer
7	7	Ben Hoffmann	Seitenstraße 7, 55257 Budenheim	24.12.1979	ben.hoffmann@example.com	015100000007	Benutzer
8	8	Emma Becker	Kapitelweg 8, 55257 Budenheim	09.09.1989	emma.becker@example.com	015100000008	Benutzer
9	9	Paul Schulz	Bücherplatz 9, 55257 Budenheim	17.11.2003	paul.schulz@example.com	015100000009	Benutzer
10	10	Sofia Richter	Leseplatz 10, 55257 Budenheim	21.06.1985	sofia.richter@example.com	015100000010	Benutzer
11	11	Tim Klein	Medienweg 11, 55257 Budenheim	03.08.1985	tim.klein@example.com	015100000011	Benutzer
12	12	Laura Wolf	Tauschstraße 12, 55257 Budenheim	18.10.1999	laura.wolf@example.com	015100000012	Benutzer

Quelle: Eigene Darstellung 6

Tabelle Genre

Speichert Buchgenres.

```
CREATE TABLE genre (
  genre_id INT AUTO_INCREMENT PRIMARY KEY,
  genrename VARCHAR(100) NOT NULL UNIQUE
);
```

```
DESCRIBE genre;
```

Abbildung 7 Genre

Field	Type	Null	Key	Default	Extra
genre_id	int(11)	NO	PRI	[NULL]	auto_increment
genrename	varchar(100)	NO	UNI	[NULL]	

Quelle: Eigene Darstellung 7

```
SELECT * from genre;
```

Abbildung 8 select Aufruf Genre

genre_id	genrename
6	Biografie
12	Comic
13	Drama
3	Fantasy
10	Jugendbuch
9	Kinderbuch
2	Krimi
11	Manga
8	Ratgeber
1	Roman
14	Romance
7	Sachbuch
4	Science Fiction

Quelle: Eigene Darstellung 8

Tabelle Autor

Speichert Autoren.

```
CREATE TABLE autor (
  autor_id INT AUTO_INCREMENT PRIMARY KEY,
  autorname VARCHAR(255) NOT NULL
);
```

```
DESCRIBE autor;
```

Abbildung 9 Autor

Field	Type	Null	Key	Default	Extra
autor_id	int(11)	NO	PRI	[NULL]	auto_increment
autorname	varchar(255)	NO		[NULL]	

Quelle: Eigene Darstellung 9

```
SELECT * from autor;
```

Abbildung 10 select Aufruf Autor

	123 autor_id	AZ autorname
1	1	Eiichiro Oda
2	2	Akira Toriyama
3	3	Masashi Kishimoto
4	4	Koyoharu Gotouge
5	5	Gege Akutami
6	6	Chugong
7	7	J. K. Rowling
8	8	Stephen King
9	9	Agatha Christie
10	10	George Orwell
11	11	J. R. R. Tolkien
12	12	Cornelia Funke

Quelle: Eigene Darstellung 10

Tabelle Sprache

Speichert Sprachen.

```
CREATE TABLE sprache (
  sprache_id INT AUTO_INCREMENT PRIMARY KEY,
  sprache VARCHAR(100) NOT NULL UNIQUE
);
```

```
DESCRIBE sprache;
```

Abbildung 11 Sprache

	Az Field	Az Type	Az Null	Az Key	Az Default	Az Extra
1	sprache_id	int(11)	NO	PRI	[NULL]	auto_increment
2	sprache	varchar(100)	NO	UNI	[NULL]	

Quelle: Eigene Darstellung 11

```
SELECT * from sprache;
```

Abbildung 12 select Aufruf Sprache

	123 sprache_id	AZ sprache
1	9	Arabisch
2	1	Deutsch
3	2	Englisch
4	6	Französisch
5	8	Italienisch
6	4	Japanisch
7	5	Koreanisch
8	10	Russisch
9	7	Spanisch
10	3	Türkisch

Quelle: Eigene Darstellung 12

Tabelle Verlag

Speichert Verlage.

```
CREATE TABLE verlag (  
    verlag_id INT AUTO_INCREMENT PRIMARY KEY,  
    verlagsname VARCHAR(255) NOT NULL  
);
```

```
DESCRIBE verlag;
```

Abbildung 13 Verlag

Field	Type	Null	Key	Default	Extra
verlag_id	int(11)	NO	PRI	[NULL]	auto_increment
verlagsname	varchar(255)	NO		[NULL]	

Quelle: Eigene Darstellung 13

```
SELECT * from verlag;
```

Abbildung 14 select Aufruf Verlag

verlag_id	verlagsname
1	Shueisha
2	Carlsen Verlag
3	Crunchyroll Manga
4	Altraverse
5	Bloomsbury
6	Heyne Verlag
7	Atlantik Verlag
8	Fischer Verlag
9	Klett-Cotta
10	Dressler Verlag

Quelle: Eigene Darstellung 14

Tabelle Standort

Speichert Abhol- und Versandstandorte.

```
CREATE TABLE standort (  
    standort_id INT AUTO_INCREMENT PRIMARY KEY,  
    ort VARCHAR(100) NOT NULL,  
    postleitzahl VARCHAR(10) NOT NULL,  
    adresse VARCHAR(255) NOT NULL  
);
```

```
DESCRIBE standort;
```

Abbildung 15 Standort

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	standort_id	int(11)	NO	PRI	[NULL]	auto_increment
2	ort	varchar(100)	NO		[NULL]	
3	postleitzahl	varchar(10)	NO		[NULL]	
4	adresse	varchar(255)	NO		[NULL]	

Quelle: Eigene Darstellung 15

Select * from standort;

Abbildung 16 select Aufruf Standort

	standort_id	ort	postleitzahl	adresse
1	1	Budenheim	55257	Musterstraße 1
2	2	Budenheim	55257	Buchweg 2
3	3	Budenheim	55257	Lesestraße 3
4	4	Budenheim	55257	Tauschallee 4
5	5	Budenheim	55257	Romanweg 5
6	6	Budenheim	55257	Bibliotheksweg 6
7	7	Budenheim	55257	Seitenstraße 7
8	8	Budenheim	55257	Kapitelweg 8
9	9	Budenheim	55257	Bücherplatz 9
10	10	Budenheim	55257	Leseplatz 10
11	11	Budenheim	55257	DHL Paketshop, Beispielstraße 11
12	12	Budenheim	55257	DHL Packstation, Beispielweg 12
13	13	Budenheim	55257	Medienweg 13

Quelle: Eigene Darstellung 16

Tabelle Buch

Speichert grundlegende Buchinformationen.

```
CREATE TABLE buch (
    buch_id INT AUTO_INCREMENT PRIMARY KEY,
    titel VARCHAR(255) NOT NULL,
    sprache_id INT NOT NULL,
    erscheinungsjahr YEAR,
    autor_id INT NOT NULL,
    genre_id INT NOT NULL,
    verlag_id INT NOT NULL,
    FOREIGN KEY (sprache_id) REFERENCES sprache(sprache_id),
    FOREIGN KEY (autor_id) REFERENCES autor(autor_id),
    FOREIGN KEY (genre_id) REFERENCES genre(genre_id),
    FOREIGN KEY (verlag_id) REFERENCES verlag(verlag_id)
);

DESCRIBE buch;
```

Abbildung 17 Buch

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	buch_id	int(11)	NO	PRI	[NULL]	auto_increment
2	titel	varchar(255)	NO		[NULL]	
3	sprache_id	int(11)	NO	MUL	[NULL]	
4	erscheinungsjahr	year(4)	YES		[NULL]	
5	autor_id	int(11)	NO	MUL	[NULL]	
6	genre_id	int(11)	NO	MUL	[NULL]	
7	verlag_id	int(11)	NO	MUL	[NULL]	

Quelle: Eigene Darstellung 17

Select * from buch

Abbildung 18 select Aufruf Buch

	id	buch_id	titel	sprache_id	erscheinungsjahr	autor_id	genre_id	verlag_id
1	1	1	One Piece Band 1	1	1997	1	11	2
2	2	2	Dragon Ball Band 1	2	1984	2	11	2
3	3	3	Naruto Band 1	3	1999	3	11	2
4	4	4	Demon Slayer Band 1	4	2016	4	11	3
5	5	5	Jujutsu Kaisen Band 1	1	2018	5	11	3
6	6	6	Solo Leveling Band 1	7	2018	6	11	4
7	7	7	Harry Potter und der Stein der Weisen	2	1997	7	3	2
8	8	8	Es	6	1986	8	5	6
9	9	9	Mord im Orient-Express	5	1934	9	2	7
10	11	10	Der Herr der Ringe	10	1954	11	3	9
11	12	8	Tintenherz	8	2003	12	8	10
12	13	9	1984	9	1949	10	7	8

Quelle: Eigene Darstellung 18

Tabelle angebot

Verbindet Buch, Anbieter und Standort.

```
CREATE TABLE angebot (
  angebots_id INT AUTO_INCREMENT PRIMARY KEY,
  buch_id INT NOT NULL,
  benutzer_id INT NOT NULL,
  standort_id INT NOT NULL,
  verfuegbarkeit ENUM('verfuegbar','reserviert','ausgeliehen','nicht_verfuegbar') NOT NULL DEFAULT 'verfuegbar',
  leihdauer INT NOT NULL,
  versandoption ENUM('ja','nein') NOT NULL DEFAULT 'nein',
  zustand VARCHAR(100),
  FOREIGN KEY (buch_id) REFERENCES buch(buch_id),
  FOREIGN KEY (benutzer_id) REFERENCES benutzer(benutzer_id),
  FOREIGN KEY (standort_id) REFERENCES standort(standort_id)
);

DESCRIBE angebot;
```


Abbildung 19 Angebot

COLUMNS 1

COLUMNS 2

SQL Terminal

DESCRIBE anbebot

Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+ Leertaste)

Gitter

Text

Record

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	angebots_id	int(11)	NO	PRI	[NULL]	auto_increment
2	buch_id	int(11)	NO	MUL	[NULL]	
3	benutzer_id	int(11)	NO	MUL	[NULL]	
4	standort_id	int(11)	NO	MUL	[NULL]	
5	verfuegbarkeit	enum('verfuegb	NO		verfuegbar	
6	leihdauer	int(11)	NO		[NULL]	
7	versandoption	enum('ja','nein')	NO		nein	
8	zustand	varchar(100)	YES		[NULL]	

Aktualisieren

Save

Ablehnen

<

Quelle: Eigene Darstellung 19

Select * from anbebot;

Abbildung 20 select Aufruf Angebot

angebot 1 X									
Select * from anbebot Geben Sie einen SQL-Ausdruck ein, um die Ergebnisse zu filtern (verwenden Sie Strg+ Leertaste).									
	Schließen	angebots_id	123 buch_id	123 benutzer_id	123 standort_id	AZ verfuegbarkeit	123 leihdauer	AZ versandoption	AZ zustand
1	13	1	1	1	1	verfuegbar	14	nein	neu
2	14	2	2	2	2	verfuegbar	21	ja	gut
3	15	3	3	2	2	reserviert	14	nein	gut
4	16	4	4	11	11	verfuegbar	10	ja	sehr gut
5	17	5	5	4	4	ausgeliehen	21	nein	gebraucht
6	18	6	6	12	12	verfuegbar	14	ja	neu
7	19	7	7	6	6	verfuegbar	30	nein	gut
8	20	8	8	7	7	nicht_verfuegbar	14	nein	gebraucht
9	21	9	9	11	11	verfuegbar	21	ja	schlecht
10	22	11	11	12	12	verfuegbar	30	ja	sehr gut
11	23	12	12	13	13	verfuegbar	14	nein	gut
12	24	13	1	11	11	verfuegbar	21	ja	neu

Quelle: Eigene Darstellung 20

Tabelle zeitslot

Speichert mögliche Abholzeiten.

```
CREATE TABLE zeitslot (
    zeitslot_id INT AUTO_INCREMENT PRIMARY KEY,
    datum DATE NOT NULL,
    anbebots_id INT NOT NULL,
    uhrzeit_von TIME NOT NULL,
    uhrzeit_bis TIME NOT NULL,
    verfuegbarkeit_abholung ENUM('verfügbar','reserviert','nicht-verfügbar') NOT NULL DEFAULT
    'verfügbar',
    FOREIGN KEY (anbebots_id) REFERENCES anbebot(anbebots_id)
);

DESCRIBE zeitslot;
```

Abbildung 21 Zeitslot

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	zeitslot_id	int(11)	NO	PRI	[NULL]	auto_increment
2	datum	date	NO		[NULL]	
3	anbots_id	int(11)	NO	MUL	[NULL]	
4	uhrzeit_von	time	NO		[NULL]	
5	uhrzeit_bis	time	NO		[NULL]	
6	verfuegbarkeit_abholung	enum('verfügbar','reserviert','nicht_verfügbar')	NO		verfügbar	

Quelle: Eigene Darstellung 21

Select from zeitslot;

Abbildung 22 select Aufruf zeitslot

	123 zeitslot_id	2 datum	123 anbots_id	2 uhrzeit_von	2 uhrzeit_bis	AZ verfuegbarkeit_abholung
1	1	2026-05-05	13	16:00:00	18:00:00	verfügbar
2	2	2026-05-06	14	10:00:00	12:00:00	reserviert
3	3	2026-05-07	15	14:00:00	16:00:00	verfügbar
4	4	2026-05-08	16	17:00:00	19:00:00	verfügbar
5	5	2026-05-09	17	15:00:00	17:00:00	reserviert
6	6	2026-05-10	18	11:00:00	13:00:00	verfügbar
7	7	2026-05-11	19	18:00:00	20:00:00	verfügbar
8	8	2026-05-12	20	09:00:00	11:00:00	nicht_verfügbar
9	9	2026-05-13	21	13:00:00	15:00:00	verfügbar
10	10	2026-05-14	22	16:30:00	18:30:00	reserviert
11	11	2026-05-15	23	12:00:00	14:00:00	verfügbar
12	12	2026-05-16	24	15:30:00	17:30:00	verfügbar

Quelle: Eigene Darstellung 22

Tabelle buchungsliste

Speichert vorgemerkte Angebote.

```
CREATE TABLE buchungsliste (
  buchungsliste_id INT AUTO_INCREMENT PRIMARY KEY,
  benutzer_id INT NOT NULL,
  anbots_id INT NOT NULL,
  hinzuefuegt_am TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (benutzer_id) REFERENCES benutzer(benutzer_id),
  FOREIGN KEY (anbots_id) REFERENCES anbot(anbots_id)
);
```

DESCRIBE buchungsliste;

Abbildung 23 Buchungsliste

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	buchungsliste_id	int(11)	NO	PRI	[NULL]	auto_increment
2	benutzer_id	int(11)	NO	MUL	[NULL]	
3	anbots_id	int(11)	NO	MUL	[NULL]	
4	hinzuefuegt_am	timestamp	NO		current_timestamp	

Quelle: Eigene Darstellung 23

Select * from buchungsliste;

Abbildung 24 select Aufruf Buchungsliste

	123 buchungsliste_id	123 benutzer_id	123 angebots_id	hinzugefuegt_am
1	1	2	13	2026-05-02 22:46:13
2	2	3	14	2026-05-02 22:46:13
3	3	4	15	2026-05-02 22:46:13
4	4	5	16	2026-05-02 22:46:13
5	5	6	17	2026-05-02 22:46:13
6	6	7	18	2026-05-02 22:46:13
7	7	8	19	2026-05-02 22:46:13
8	8	9	20	2026-05-02 22:46:13
9	9	10	21	2026-05-02 22:46:13
10	10	11	22	2026-05-02 22:46:13
11	11	12	23	2026-05-02 22:46:13
12	12	1	24	2026-05-02 22:46:13

Quelle: Eigene Darstellung 24

Tabelle reservierung

Speichert Reservierungen.

```
CREATE TABLE reservierung (  
    reservierung_id INT AUTO_INCREMENT PRIMARY KEY,  
    buchungsliste_id INT NOT NULL,  
    reservierungsdatum DATE NOT NULL,  
    reservierungsstatus ENUM('offen','bestätigt','abgelehnt','storniert') NOT NULL DEFAULT 'offen',  
    FOREIGN KEY (buchungsliste_id) REFERENCES buchungsliste(buchungsliste_id)  
);
```

DESCRIBE reservierung;

Abbildung 25 reservierung

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	reservierung_id	int(11)	NO	PRI	[NULL]	auto_increment
2	buchungsliste_id	int(11)	NO	MUL	[NULL]	
3	reservierungsdatum	date	NO		[NULL]	
4	reservierungsstatus	enum('offen','bestätigt','abgelehnt','storniert')	NO		offen	

Quelle: Eigene Darstellung 25

select * from reservierung;

Abbildung 26 select Aufruf Reservierung

	123 reservierung_id	123 buchungsliste_id	reservierungsdatum	AZ reservierungsstatus
1	1	1	2026-05-05	bestätigt
2	2	2	2026-05-06	bestätigt
3	3	3	2026-05-07	bestätigt
4	4	4	2026-05-08	bestätigt
5	5	5	2026-05-09	bestätigt
6	6	6	2026-05-10	abgelehnt
7	7	7	2026-05-11	bestätigt
8	8	8	2026-05-12	storniert
9	9	9	2026-05-13	bestätigt
10	10	10	2026-05-14	bestätigt
11	11	11	2026-05-15	bestätigt
12	12	12	2026-05-16	bestätigt

Quelle: Eigene Darstellung 26

Tabelle ausleihe

Speichert Ausleihvorgänge.

```
CREATE TABLE ausleihe (  
    ausleihe_id INT AUTO_INCREMENT PRIMARY KEY,  
    reservierung_id INT NOT NULL,  
    ausleihdatum DATE NOT NULL,  
    rueckgabedatum DATE,  
    status ENUM('aktiv','zurückgegeben','überfällig','storniert') NOT NULL DEFAULT 'aktiv',  
    FOREIGN KEY (reservierung_id) REFERENCES reservierung(reservierung_id)  
);  
  
DESCRIBE ausleihe;
```

Abbildung 27 Ausleihe

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	ausleihe_id	int(11)	NO	PRI	[NULL]	auto_increment
2	reservierung_id	int(11)	NO	MUL	[NULL]	
3	ausleihdatum	date	NO		[NULL]	
4	rueckgabedatum	date	YES		[NULL]	
5	status	enum('aktiv','zu	NO		aktiv	

Quelle: Eigene Darstellung 27

```
SELECT * from ausleihe;
```

Abbildung 28 select Aufruf Ausleihe

	ausleihe_id	reservierung_id	ausleihdatum	rueckgabedatum	AZ status
1	1	1	2026-05-06	2026-05-17	zurückgegeben
2	2	2	2026-05-07	2026-05-21	aktiv
3	3	3	2026-05-08	2026-05-22	aktiv
4	4	4	2026-05-09	2026-05-23	zurückgegeben
5	5	5	2026-05-10	2026-05-24	aktiv
6	6	7	2026-05-12	2026-05-26	zurückgegeben
7	7	9	2026-05-14	2026-05-28	aktiv
8	8	10	2026-05-15	2026-05-29	aktiv
9	9	11	2026-05-16	2026-05-30	zurückgegeben
10	10	12	2026-05-17	2026-05-31	aktiv

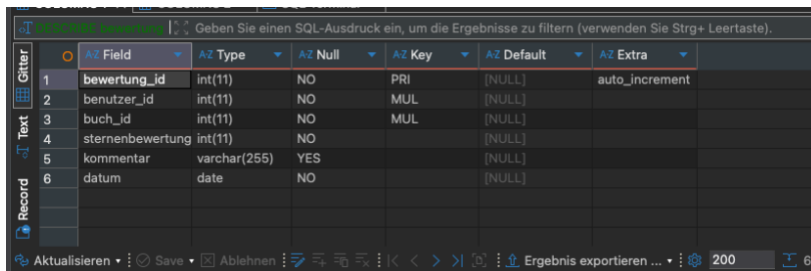
Quelle: Eigene Darstellung 28

Tabelle bewertung

Speichert Bewertungen.

```
CREATE TABLE bewertung (  
    bewertung_id INT AUTO_INCREMENT PRIMARY KEY,  
    benutzer_id INT NOT NULL,  
    buch_id INT NOT NULL,  
    sternenbewertung INT NOT NULL,  
    kommentar VARCHAR(255),  
    datum DATE NOT NULL,  
    FOREIGN KEY (benutzer_id) REFERENCES benutzer(benutzer_id),  
    FOREIGN KEY (buch_id) REFERENCES buch(buch_id),  
    CHECK (sternenbewertung BETWEEN 1 AND 5)  
);  
  
DESCRIBE bewertung;
```

Abbildung 29 Bewertung

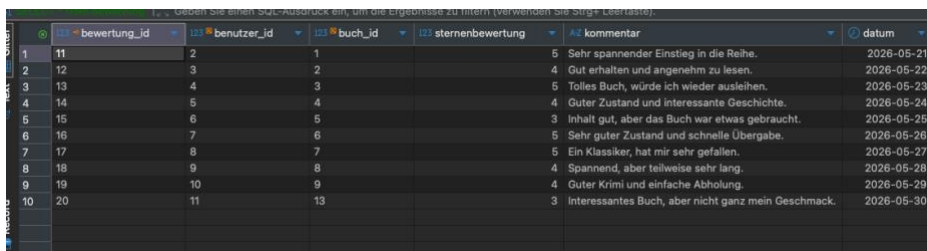


	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	bewertung_id	int(11)	NO	PRI	[NULL]	auto_increment
2	benutzer_id	int(11)	NO	MUL	[NULL]	
3	buch_id	int(11)	NO	MUL	[NULL]	
4	sternenbewertung	int(11)	NO		[NULL]	
5	kommentar	varchar(255)	YES		[NULL]	
6	datum	date	NO		[NULL]	

Quelle: Eigene Darstellung 29

SELECT * from bewertung

Abbildung 30 select Aufruf Bewertung



	11: bewertung_id	103: benutzer_id	103: buch_id	103: sternenbewertung	A2: kommentar	datum
1	11	2	1	5	Sehr spannender Einstieg in die Reihe.	2026-05-21
2	12	3	2	4	Gut erhalten und angenehm zu lesen.	2026-05-22
3	13	4	3	5	Tolles Buch, würde ich wieder ausleihen.	2026-05-23
4	14	5	4	4	Guter Zustand und interessante Geschichte.	2026-05-24
5	15	6	5	3	Inhalt gut, aber das Buch war etwas gebraucht.	2026-05-25
6	16	7	6	5	Sehr guter Zustand und schnelle Übergabe.	2026-05-26
7	17	8	7	5	Ein Klassiker, hat mir sehr gefallen.	2026-05-27
8	18	9	8	4	Spannend, aber teilweise sehr lang.	2026-05-28
9	19	10	9	4	Guter Krimi und einfache Abholung.	2026-05-29
10	20	11	13	3	Interessantes Buch, aber nicht ganz mein Geschmack.	2026-05-30

Quelle: Eigene Darstellung 30

Tabelle zahlung

Speichert Zahlungsdaten.

```
CREATE TABLE zahlung (  
    zahlungs_id INT AUTO_INCREMENT PRIMARY KEY,  
    ausleihe_id INT NOT NULL,  
    zahlungsoption ENUM('bar','paypal','überweisung','karte') NOT NULL,  
    betrag DECIMAL(10,2) NOT NULL,  
    zahlungsstatus ENUM('offen','bezahlt','storniert') NOT NULL DEFAULT 'offen',
```

```
FOREIGN KEY (ausleihe_id) REFERENCES ausleihe(ausleihe_id)
);

DESCRIBE zahlung;
```

Abbildung 31 Zahlung

AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1 zahlungs_id	int(11)	NO	PRI	[NULL]	auto_increment
2 ausleihe_id	int(11)	NO	MUL	[NULL]	
3 zahlungsoption	enum('bar','paypal','überweisung')	NO		[NULL]	
4 betrag	decimal(10,2)	NO		[NULL]	
5 zahlungsstatus	enum('offen','bezahlt')	NO		offen	

Quelle: Eigene Darstellung 31

```
SELECT * FROM zahlung;
```

Abbildung 32 select Aufruf Zahlung

zahlungs_id	ausleihe_id	zahlungsoption	betrag	zahlungsstatus
1	1	bar	0	bezahlt
2	2	paypal	2,5	bezahlt
3	3	überweisung	1,5	offen
4	4	karte	3	bezahlt
5	5	bar	0	offen
6	6	paypal	2	bezahlt
7	7	überweisung	1	bezahlt
8	8	karte	2,5	offen
9	9	bar	0	bezahlt
10	10	paypal	3,5	bezahlt

Quelle: Eigene Darstellung 32

Tabelle rechnung

Speichert Rechnungen.

```
CREATE TABLE rechnung (
  rechnungs_id INT AUTO_INCREMENT PRIMARY KEY,
  zahlungs_id INT NOT NULL,
  rechnungsdatum DATE NOT NULL,
  rechnungsname VARCHAR(255) NOT NULL,
  rechnungsadresse VARCHAR(255) NOT NULL,
  betrag DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (zahlungs_id) REFERENCES zahlung(zahlungs_id)
);

DESCRIBE rechnung;
```

Abbildung 33 Rechnung

AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1 rechnungs_id	int(11)	NO	PRI	[NULL]	auto_increment
2 zahlungs_id	int(11)	NO	MUL	[NULL]	
3 rechnungsdatum	date	NO		[NULL]	
4 rechnungsname	varchar(255)	NO		[NULL]	
5 rechnungsadresse	varchar(255)	NO		[NULL]	
6 betrag	decimal(10,2)	NO		[NULL]	

Quelle: Eigene Darstellung 33

SELECT * FROM rechnung;

Abbildung 34 select Aufruf Rechnung

	123 rechnungs_id	123 zahlungs_id	rechnungsdatum	AZ rechnungsname	AZ rechnungsadresse	123 betrag
1	1		2026-05-06	Anna Beispiel	Buchweg 2, 55257 Budenheim	0
2	2	2	2026-05-07	Lukas Weber	Lesestraße 3, 55257 Budenheim	2,5
3	3	3	2026-05-08	Mia Schneider	Tauschallee 4, 55257 Budenheim	1,5
4	4	4	2026-05-09	Noah Fischer	Romanweg 5, 55257 Budenheim	3
5	5	5	2026-05-10	Lea Wagner	Bibliotheksweg 6, 55257 Budenheim	0
6	6	6	2026-05-12	Ben Hoffmann	Seitenstraße 7, 55257 Budenheim	2
7	7	7	2026-05-14	Emma Becker	Kapitelweg 8, 55257 Budenheim	1
8	8	8	2026-05-15	Paul Schulz	Bücherplatz 9, 55257 Budenheim	2,5
9	9	9	2026-05-16	Sofia Richter	Leseplatz 10, 55257 Budenheim	0
10	10	10	2026-05-17	Tim Klein	Medienweg 11, 55257 Budenheim	3,5

Quelle: Eigene Darstellung 34

Dummy-Daten und Datenkonsistenz

Nach der Erstellung der Tabellen wurden Dummy-Daten eingefügt, um sicherzustellen, dass jede Tabelle mindestens zehn Datensätze enthält. Dies ermöglicht realistische Tests der Datenbankfunktionalität. Die Dummy-Daten basieren auf einer lokalen Buchtausch-App in Budenheim und umfassen Benutzer, Bücher, Standorte, Angebote, Reservierungen, Ausleihen, Bewertungen, Zahlungen und Rechnungen.

```
SELECT 'benutzer' AS tabelle, COUNT(*) AS anzahl FROM benutzer
UNION ALL SELECT 'genre', COUNT(*) FROM genre
UNION ALL SELECT 'autor', COUNT(*) FROM autor
UNION ALL SELECT 'sprache', COUNT(*) FROM sprache
UNION ALL SELECT 'verlag', COUNT(*) FROM verlag
UNION ALL SELECT 'standort', COUNT(*) FROM standort
UNION ALL SELECT 'buch', COUNT(*) FROM buch
UNION ALL SELECT 'angebot', COUNT(*) FROM anbot
UNION ALL SELECT 'zeitslot', COUNT(*) FROM zeitslot
UNION ALL SELECT 'buchungsliste', COUNT(*) FROM buchungsliste
UNION ALL SELECT 'reservierung', COUNT(*) FROM reservierung
UNION ALL SELECT 'ausleihe', COUNT(*) FROM ausleihe
UNION ALL SELECT 'bewertung', COUNT(*) FROM bewertung
```

```

UNION ALL SELECT 'zahlung', COUNT(*) FROM zahlung
UNION ALL SELECT 'rechnung', COUNT(*) FROM rechnung;

```

Abbildung 35 Dummy-Daten und Datenkonsistenz

	A-Z tabelle	123 anzahl
1	benutzer	12
2	genre	15
3	autor	12
4	sprache	10
5	verlag	10
6	standort	13
7	buch	12
8	anbot	12
9	zeitslot	12
10	buchungsliste	12
11	reservierung	12
12	ausleihe	10
13	bewertung	10

Quelle: Eigene Darstellung 35

Bei den Angebotsdaten wurde zusätzlich geprüft, ob Angebote mit Versandoption passenden DHL-Standorten zugeordnet sind und ob Angebote ohne Versandoption die Adresse des jeweiligen Anbieters verwenden. Dafür wurde unter anderem die Funktion CONCAT() eingesetzt, um die aufgeteilten Standortdaten mit der vollständigen Benutzeradresse vergleichen zu können.

```

UPDATE anbot ag
JOIN benutzer u ON ag.benutzer_id = u.benutzer_id
JOIN standort s ON CONCAT(s.adresse, ' ', s.postleitzahl, ' ', s.ort) = u.adresse
SET ag.standort_id = s.standort_id
WHERE ag.versandoption = 'nein';

```


Abbildung 36 Update Angebotsdaten

Statistics 1 X	
Name	Value
Updated Rows	12
Execute time	0,054s
Start time	Sun May 31 00:26:36 CEST 2026
Finish time	Sun May 31 00:26:36 CEST 2026
Query	UPDATE anbot ag JOIN benutzer u ON ag.benutzer_id = u.benutzer_id JOIN standort s ON CONCAT(s.adresse, ' ', s.postleitzahl, ' ', s.ort) = u.adresse SET ag.standort_id = s.standort_id

Quelle: Eigene Darstellung 36

Testfälle und Kontrollabfragen

Testfall 1 – Anzeige verfügbarer Buchangebote

Ziel: Mit diesem Testfall wird überprüft, ob verfügbare Buchangebote korrekt zusammen mit den zugehörigen Buchdaten, Benutzerdaten und Standortinformationen angezeigt werden. Dabei wird getestet, ob die Beziehungen zwischen den Tabellen anbot, buch, benutzer und standort korrekt funktionieren.

Durchführung: Es wird eine SQL-Abfrage ausgeführt, die alle verfügbaren Angebote ausgibt. Zusätzlich werden Titel des Buches, Anbieter, Abholort, Leihdauer, Versandoption und Zustand angezeigt.

```
SELECT
    ag.angebots_id,
    b.titel,
    u.name AS anbieter,
    CONCAT(s.adresse, ' ', s.postleitzahl, ' ', s.ort) AS abholort,
    ag.verfuegbarkeit,
    ag.leihdauer,
    ag.versandoption,
    ag.zustand
FROM anbot ag
JOIN buch b ON ag.buch_id = b.buch_id
JOIN benutzer u ON ag.benutzer_id = u.benutzer_id
JOIN standort s ON ag.standort_id = s.standort_id
WHERE ag.verfuegbarkeit = 'verfuegbar'
ORDER BY b.titel;
```

Ergebnis: Die Abfrage liefert erfolgreich alle verfügbaren Buchangebote mit den dazugehörigen Informationen. Dadurch wird bestätigt, dass die Tabellen korrekt miteinander verknüpft wurden und die Such- und Filterfunktion der Datenbank ordnungsgemäß funktioniert.

Abbildung 37 Test 1

	123	angebots_id	A2	titel	A2	anbieter	A2	abholort	A2	verfuegbarkeit	123	leihdauer	A2	versandoption	A2	zustand
1				1984		Max Mustermann		Musterstraße 1, 55257 Budenheim		verfuegbar		21	ja		neu	
2		16		Demon Slayer Band 1		Mia Schneider		Lesestraße 3, 55257 Budenheim		verfuegbar		10	ja		sehr gut	
3		22		Der Herr der Ringe		Tim Klein		Leseplatz 10, 55257 Budenheim		verfuegbar		30	ja		sehr gut	
4		14		Dragon Ball Band 1		Anna Beispiel		Buchweg 2, 55257 Budenheim		verfuegbar		21	ja		gut	
5		19		Harry Potter und der Stein der Weisen		Ben Hoffmann		Bibliotheksweg 6, 55257 Budenheim		verfuegbar		30	nein		gut	
6		21		Mord im Orient-Express		Paul Schulz		Kapitelweg 8, 55257 Budenheim		verfuegbar		21	ja		schlecht	
7		13		One Piece Band 1		Max Mustermann		Musterstraße 1, 55257 Budenheim		verfuegbar		14	nein		neu	
8		18		Solo Leveling Band 1		Lea Wagner		Romanweg 5, 55257 Budenheim		verfuegbar		14	ja		neu	
9		23		Tintenherz		Laura Wolf		Medienweg 13, 55257 Budenheim		verfuegbar		14	nein		gut	

Quelle: Eigene Darstellung 37

Testfall 2 – Nachverfolgung des Ausleih- und Zahlungsprozesses

Ziel: Mit diesem Testfall wird überprüft, ob der vollständige Ausleihprozess korrekt abgebildet wird. Dabei wird getestet, ob Buchungsliste, Reservierung, Ausleihe, Zahlung und Rechnung richtig miteinander verknüpft sind.

Durchführung: Es wird eine SQL-Abfrage ausgeführt, die die Daten aus mehreren Tabellen kombiniert und den gesamten Ablauf eines Buchausleihprozesses darstellt.

```

SELECT
    bl.buchungsliste_id,
    r.reservierung_id,
    a.ausleihe_id,
    z.zahlungs_id,
    re.rechnungs_id,
    u.name AS ausleiher,
    b.titel AS buch,
    r.reservierungsstatus,
    a.status AS ausleihstatus,
    z.zahlungsstatus,
    re.betrag AS rechnungsbetrag
FROM buchungsliste bl
LEFT JOIN reservierung r ON bl.buchungsliste_id = r.buchungsliste_id
LEFT JOIN ausleihe a ON r.reservierung_id = a.reservierung_id
LEFT JOIN zahlung z ON a.ausleihe_id = z.ausleihe_id
LEFT JOIN rechnung re ON z.zahlungs_id = re.zahlungs_id
JOIN benutzer u ON bl.benutzer_id = u.benutzer_id
JOIN angebot ag ON bl.angebots_id = ag.angebots_id
JOIN buch b ON ag.buch_id = b.buch_id
ORDER BY bl.buchungsliste_id;

```

Ergebnis: Die Abfrage liefert erfolgreich die zusammengehörigen Daten aus allen beteiligten Tabellen. Dadurch wird bestätigt, dass die Beziehungen zwischen Buchungsliste, Reservierung, Ausleihe, Zahlung und Rechnung korrekt implementiert wurden und der vollständige Ausleihprozess innerhalb der Datenbank nachvollziehbar dargestellt werden kann.

Abbildung 38 Test 2

	101 buchungsliste_id	102 reservierung_id	103 ausleihe_id	104 zahlungs_id	105 rechnungs_id	A1 ausleiher	A1 buch	A2 reservierungsstatus	A2 ausleihstatus	A1 zahlungsstatus	107 rechnungsbetrag
1	1	1	1	1	1	Anna Beispiel	One Piece Band 1	bestätigt	zurückgegeben	bezahlt	0
2	2	2	2	2	2	Lukas Weber	Dragon Ball Band 1	bestätigt	aktiv	bezahlt	2,6
3	3	3	3	3	3	Mia Schneider	Naruto Band 1	bestätigt	aktiv	offen	1,5
4	4	4	4	4	4	Noah Fischer	Demon Slayer Band 1	bestätigt	zurückgegeben	bezahlt	3
5	5	5	5	5	5	Lea Wagner	Jujutsu Kaisen Band 1	bestätigt	aktiv	offen	0
6	6	6	[NULL]	[NULL]	[NULL]	Ben Hoffmann	Solo Leveling Band 1	abgelehnt	[NULL]	[NULL]	[NULL]
7	7	7	6	6	6	Emma Becker	Harry Potter und der Stein der Weisen	bestätigt	zurückgegeben	bezahlt	2
8	8	8	[NULL]	[NULL]	[NULL]	Paul Schutz	Es	storniert	[NULL]	[NULL]	[NULL]
9	9	9	7	7	7	Sofia Richter	Mord im Orient-Express	bestätigt	aktiv	bezahlt	1
10	10	10	8	8	8	Tim Klein	Der Herr der Ringe	bestätigt	aktiv	offen	2,5
11	11	11	9	9	9	Laura Wolf	Tintenherz	bestätigt	zurückgegeben	bezahlt	0
12	12	12	10	10	10	Max Mustermann	1984	bestätigt	aktiv	bezahlt	3,5

Quelle: Eigene Darstellung 38

Testfall 3 – Anzeige von Bewertungen

Ziel: Dieser Testfall prüft, ob Bewertungen korrekt mit Benutzer und Buch verknüpft werden.

Durchführung: Es wird eine SQL-Abfrage ausgeführt, die alle vorhandenen Bewertungen ausgibt. Zusätzlich werden der Name des bewertenden Benutzers, der Titel des bewerteten Buches, die Sternebewertung, der Kommentar und das Bewertungsdatum angezeigt.

```
SELECT
    bw.bewertung_id,
    u.name AS benutzer,
    b.titel AS bewertetes_buch,
    bw.sternenbewertung,
    bw.kommentar,
    DATE_FORMAT(bw.datum, '%d.%m.%Y') AS datum
FROM bewertung bw
JOIN benutzer u ON bw.benutzer_id = u.benutzer_id
JOIN buch b ON bw.buch_id = b.buch_id
ORDER BY bw.bewertung_id;
```

Ergebnis: Die Abfrage liefert erfolgreich alle Bewertungen mit den dazugehörigen Benutzer- und Buchinformationen. Dadurch wird bestätigt, dass Bewertungen korrekt mit Benutzern und Büchern verknüpft wurden und die Bewertungsfunktion der Datenbank ordnungsgemäß funktioniert.

Abbildung 39 Test 3

	123 bewertung_id	A2 benutzer	A2 bewertetes_buch	123 sternenbewertung	A2 kommentar	A2 datum
1	11	Furkan Demir	One Piece Band 1	5	Sehr spannender Einstieg in die Reihe.	21.05.2026
2	12	Emirhan Demir	Dragon Ball Band 1	4	Gut erhalten und angenehm zu lesen.	22.05.2026
3	13	Aydin Ünal	Naruto Band 1	5	Tolles Buch, würde ich wieder ausleihen.	23.05.2026
4	14	Damla Ünal	Demon Slayer Band 1	4	Guter Zustand und interessante Geschichte.	24.05.2026
5	15	Ugur Gursoy	Jujutsu Kaisen Band 1	3	Inhalt gut, aber das Buch war etwas gebraucht.	25.05.2026
6	16	Ahmet Balakci	Solo Leveling Band 1	5	Sehr guter Zustand und schnelle Übergabe.	26.05.2026
7	17	Edwin Brons	Harry Potter und der Stein der Weisen	5	Ein Klassiker, hat mir sehr gefallen.	27.05.2026
8	18	Annisa Demir	Es	4	Spannend, aber teilweise sehr lang.	28.05.2026
9	19	Irmak Kordel	Mord im Orient-Express	4	Guter Krimi und einfache Abholung.	29.05.2026
10	20	Burcu Kabacam	1984	3	Interessantes Buch, aber nicht ganz mein Geschmack.	30.05.2026

Quelle: Eigene Darstellung 39

Testfall 4 – Anzeige von Zahlungen

Ziel: Dieser Testfall prüft, ob Zahlungen einer Ausleihe, einem Benutzer und einem Buch zugeordnet werden können.

Durchführung: Es wird eine SQL-Abfrage ausgeführt, die alle Zahlungsdaten ausgibt. Zusätzlich werden die zugehörige Ausleihe, der Name des Ausleihers, der Buchtitel, die Zahlungsoption, der Betrag und der Zahlungsstatus angezeigt.

```
SELECT
    z.zahlungs_id,
    z.ausleihe_id,
    u.name AS ausleiher,
    b.titel AS buch,
    z.zahlungsoption,
    z.betrag,
    z.zahlungsstatus
FROM zahlung z
JOIN ausleihe a ON z.ausleihe_id = a.ausleihe_id
JOIN reservierung r ON a.reservierung_id = r.reservierung_id
JOIN buchungsliste bl ON r.buchungsliste_id = bl.buchungsliste_id
JOIN benutzer u ON bl.benutzer_id = u.benutzer_id
JOIN angebot ag ON bl.angebots_id = ag.angebots_id
JOIN buch b ON ag.buch_id = b.buch_id
ORDER BY z.zahlungs_id;
```

Ergebnis: Die Abfrage liefert erfolgreich alle Zahlungsdaten mit den dazugehörigen Informationen zum Ausleiher und zum Buch. Dadurch wird bestätigt, dass Zahlungen korrekt in den Ausleihprozess eingebunden wurden und die Beziehungen zwischen den beteiligten Tabellen funktionieren.

Abbildung 40 Test 4

	123 zahlungs_id	123 ausleihe_id	AZ ausleiher	AZ buch	AZ zahlungsoption	123 betrag	AZ zahlungsstatus
1	1	1	Anna Beispiel	One Piece Band 1	bar	0	bezahlt
2	2	2	Lukas Weber	Dragon Ball Band 1	paypal	2,5	bezahlt
3	3	3	Mia Schneider	Naruto Band 1	überweisung	1,5	offen
4	4	4	Noah Fischer	Demon Slayer Band 1	karte	3	bezahlt
5	5	5	Lea Wagner	Jujutsu Kaisen Band 1	bar	0	offen
6	6	6	Emma Becker	Harry Potter und der Stein der Weisen	paypal	2	bezahlt
7	7	7	Sofia Richter	Mord im Orient-Express	überweisung	1	bezahlt
8	8	8	Tim Klein	Der Herr der Ringe	karte	2,5	offen
9	9	9	Laura Wolf	Tintenherz	bar	0	bezahlt
10	10	10	Max Mustermann	1984	paypal	3,5	bezahlt

Quelle: Eigene Darstellung 40

Testfall 5 – Aktualisierung einer Ausleihe

Ziel: Dieser Testfall prüft, ob der Status einer bestehenden Ausleihe aktualisiert und eine Rückgabe dokumentiert werden kann.

Durchführung: Der Status einer bestehenden Ausleihe wird mit einer UPDATE-Anweisung auf zurückgegeben gesetzt. Zusätzlich wird das Rückgabedatum mit dem aktuellen Datum eingetragen. Anschließend wird der geänderte Datensatz mit einer SELECT-Abfrage kontrolliert.

```
UPDATE ausleihe
SET status = 'zurückgegeben',
    rueckgabedatum = CURDATE()
```

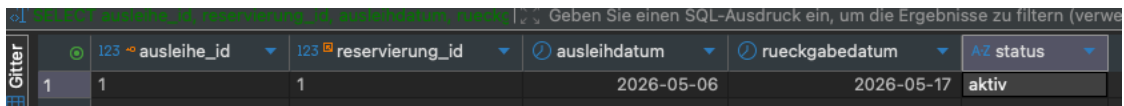
```
WHERE ausleihe_id = 1;
```

```
SELECT
    ausleihe_id,
    reservierung_id,
    ausleihdatum,
    rueckgabedatum,
    status
FROM ausleihe
WHERE ausleihe_id = 1;
```

Erwartetes Ergebnis: Die Änderung wurde erfolgreich übernommen und die betroffene Ausleihe wird mit dem Status zurückgegeben sowie dem eingetragenen Rückgabedatum angezeigt. Dadurch wird bestätigt, dass Aktualisierungen innerhalb der Tabelle ausleihe korrekt funktionieren.

Vorher

Abbildung 41 Test 5 Vorher

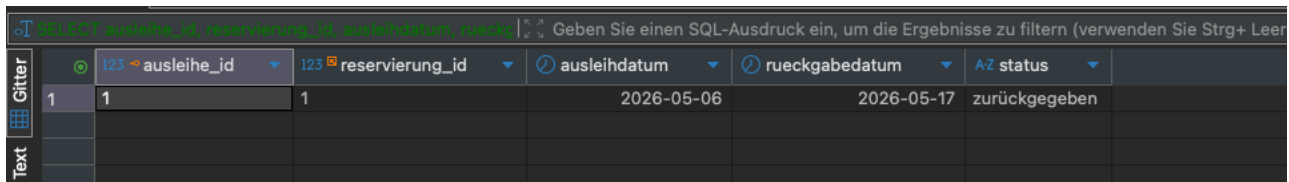


	123 ausleihe_id	123 reservierung_id	ausleihdatum	rueckgabedatum	AZ status
1	1	1	2026-05-06	2026-05-17	aktiv

Quelle: Eigene Darstellung 41

Nachher

Abbildung 42 Test 5 Nachher



	123 ausleihe_id	123 reservierung_id	ausleihdatum	rueckgabedatum	AZ status
1	1	1	2026-05-06	2026-05-17	zurückgegeben

Quelle: Eigene Darstellung 42

Probleme, Fehleranalyse und Lösungen

Im Verlauf der Implementierung traten standardmäßige Datenbankfehler auf. Diese Fehler erwiesen sich als vorteilhaft, da sie die korrekte Funktionsweise der Fremdschlüssel und der Syntaxregeln des Datenbankmanagementsystems bestätigten.

Tabelle 4 Probleme, Fehleranalyse

Problem	Ursache	Lösung
Fremdschlüssel-Fehler bei angebot oder bewertung	Es wurde auf eine buch_id verwiesen, die nicht mehr in der Tabelle buch existierte.	Die betroffenen INSERT- Statements wurden angepasst und nur vorhandene Buch-IDs verwendet.

Unlogische Standortzuordnung	Benutzeradresse und Standortadresse waren unterschiedlich formatiert.	Mit CONCAT() wurden Standortadresse, Postleitzahl und Ort zu einer vollständigen Adresse zusammengesetzt.
------------------------------	---	---

Zusammenfassung der Implementierung

In der Erarbeitungs- und Reflexionsphase wurde das zuvor entwickelte ER-Modell der Buchtausch-App in eine relationale SQL-Datenbank umgesetzt. Während der Umsetzung wurde zunächst ein Entwicklungsskript genutzt, um Tabellen, Testdaten, Korrekturen und Kontrollabfragen schrittweise zu erarbeiten. Für die Abgabe wurde dieses Arbeitsskript bereinigt und in ein strukturiertes SQL-Skript überführt. Dadurch sind die Datenbankerstellung, die Tabellenerstellung, das Einfügen der Dummy-Daten sowie die Testabfragen übersichtlich und nachvollziehbar dokumentiert.

Dafür wurde zunächst die Datenbank buchtausch_app erstellt. Der Zeichensatz utf8mb4 wurde verwendet, damit auch Umlaute, Sonderzeichen und internationale Zeichen korrekt gespeichert werden können.

Die Tabellen wurden in einer fachlich sinnvollen Reihenfolge angelegt. Zuerst wurden die Stammdatentabellen erstellt, zum Beispiel benutzer, genre, autor, sprache, verlag und standort. Danach folgten die abhängigen Tabellen wie buch, anbot, zeitslot, buchungsliste, reservierung, ausleihe, bewertung, zahlung und rechnung. Diese Reihenfolge war wichtig, da viele Tabellen über Fremdschlüssel auf bereits bestehende Tabellen verweisen.

Die Beziehungen zwischen den Tabellen wurden durch Primär- und Fremdschlüssel abgesichert. Dadurch wird die referenzielle Integrität der Datenbank gewährleistet. Ein Angebot kann beispielsweise nur zu einem vorhandenen Buch, Benutzer und Standort gespeichert werden. Ebenso kann eine Ausleihe nur aus einer bestehenden Reservierung entstehen und eine Rechnung nur einer vorhandenen Zahlung zugeordnet werden. Zusätzlich wurden Einschränkungen wie UNIQUE für eindeutige E-Mail-Adressen und CHECK für gültige Sternbewertungen eingesetzt.

Nach der Erstellung der Tabellen wurden sinnvolle Dummy-Daten eingefügt. Diese Testdaten bilden den Ablauf der Buchtausch-App realistisch ab: Benutzer können Bücher anbieten, Angebote können in eine Buchungsliste aufgenommen, reserviert und ausgeliehen werden. Außerdem werden Bewertungen, Zahlungen und Rechnungen dokumentiert. Damit konnte geprüft werden, ob die Datenbank nicht nur technisch funktioniert, sondern auch den geplanten Geschäftsprozess korrekt unterstützt.

Zur Kontrolle der Umsetzung wurden mehrere SQL-Testfälle mit JOIN-Abfragen durchgeführt. Dabei wurden unter anderem verfügbare Buchangebote, Bewertungen, Zahlungen sowie der vollständige Prozess von der Buchungsliste bis zur Rechnung überprüft. Zusätzlich wurden fehlerhafte oder uneinheitliche Daten, zum Beispiel bei Standort- und Adresszuordnungen, durch gezielte UPDATE-Anweisungen korrigiert.

Die Implementierung zeigt, dass das Datenbankmodell der Buchtausch-App funktionsfähig, normalisiert und nachvollziehbar ist. Die Tabellen sind logisch verbunden, die Dummy-Daten ermöglichen realistische Tests und die SQL-Abfragen zeigen, dass zentrale Anwendungsfälle abgebildet werden können.

Fazit

In der zweiten Phase wurde das ER-Modell aus der Konzeptionsphase in einer relationalen SQL-Datenbank umgesetzt. Alle Tabellen mit Primär- und Fremdschlüsseln sowie Datentypen wurden erstellt. Die Datenbank bildet den gesamten Ablauf einer Buchtausch-App ab: Benutzer, Bücher, Angebote, Standorte, Zeitslots, Buchungsliste, Reservierung, Ausleihe, Bewertung, Zahlung und Rechnung.

Besonders wichtig war die korrekte Umsetzung der Tabellenbeziehungen. Fremdschlüssel stellen sicher, dass Datensätze nur gespeichert werden können, wenn die übergeordneten Daten vorhanden sind. Eine Ausleihe braucht eine Reservierung, eine Rechnung eine Zahlung. So bleibt die referenzielle Integrität gewahrt.

Die Datenbank enthält sinnvolle Dummy-Daten für realistische Tests. Für jede Tabelle gibt es genug Beispieldaten, um Beziehungen und Abfragen klar darzustellen. Wir haben typische Datenbankprobleme wie fehlerhafte Fremdschlüssel und uneinheitliche Adressen mit Korrekturen, UPDATE-Anweisungen und Kontrollabfragen behoben.

Mehrere Testfälle überprüften die Datenbank, darunter Buchangebote, Ausleihen, Zahlungen, Bewertungen und Statusänderungen. Die Ergebnisse zeigen, dass die Tabellen korrekt verknüpft sind und die wichtigsten Geschäftsprozesse abbilden.

In Phase 2 implementierten wir eine normalisierte Datenbankstruktur. Wir dokumentierten die SQL-Statements, füllten die Tabellen mit Testdaten und überprüften sie mit Abfragen.